

과제 10

AI와 함께 쓰는 첫 논문

프롬프트의 구체성이 AI 생성 코드의 정확도에 미치는 영향

공개 실험 데이터 재분석 연구

작성자: 서효상

작성일: 2026년 9월 15일

관심 분야: 생성형 AI · 코딩 · 프롬프트 엔지니어링 · 소프트웨어 품질

초록

본 연구는 프롬프트의 구체성이 AI가 생성한 코드의 기능적 정확도에 어떤 영향을 주는지 확인하였다. Murr, Grainger, Gao(2023)의 공개 코드 생성 실험 집계 데이터를 2026년 9월 15일에 수집하여 재분석하였다. 5개 언어모델의 네 가지 프롬프트 유형 가운데, 가장 모호한 ‘함수명이 가려진 테스트만 제공’ 조건과 문제 설명 및 명시적 테스트를 함께 제공한 고구체성 조건을 핵심 비교 대상으로 삼았다. 정확도는 테스트 통과율로 측정하였다. 저구체성 조건은 200/315(63.49%), 고구체성 조건은 228/315(72.38%)로 고구체성 조건이 8.89%p 높았다. 5개 모델 가운데 4개에서는 통과율이 증가했지만 Text-davinci-002에서는 7.94%p 감소했다. 따라서 구체적인 프롬프트는 전체적으로 코드 정확도를 높이는 방향과 관련되었으나, 모델에 따라 효과가 다를 수 있다는 결론에 도달하였다. 이 결과는 AI 코딩에서 요구사항, 입력·출력, 테스트 예시를 명확히 제시하되 생성 결과를 실제 테스트로 검증해야 함을 보여준다.

핵심어: 생성형 AI, 코드 생성, 프롬프트 구체성, 프롬프트 엔지니어링, 테스트 통과율

1. 서론: 관심 분야에서 가설까지

1.1 관심 분야

관심 분야는 생성형 AI, 코딩, 프롬프트 작성 방법, 소프트웨어 오류 감소이다. 수업에서 AI를 활용해 코드를 작성하고 오류를 해결하는 경험이 있었기 때문에 “AI에게 어떻게 질문하느냐가 코드 결과에 영향을 주는가”를 연구 질문으로 발전시켰다.

1.2 검토한 후보 주제와 제외 이유

후보 주제	측정 가능성	판단 및 제외 이유
AI 사용 여부가 코딩 과제 완료 시간에 미치는 영향	시간 측정 가능	같은 난이도의 과제를 반복하기 어렵고 학습효과가 개입될 수 있어 제외
AI 사용이 코드 이해도에 미치는 영향	시험 점수로 측정 가능	이해도 문항을 새로 설계해야 하고 평가 기준의 주관성이 커질 수 있어 제외
프롬프트의 구체성이 AI 생성 코드 정확도에 미치는 영향	테스트 통과/실패로 객관적 측정 가능	공개 실험 자료가 있고 재현성이 높아 최종 선택
AI 사용이 디버깅 시도 횟수에 미치는 영향	수정 횟수 기록 가능	오류 종류와 난이도를 동일하게 맞추기 어렵고 직접 실험 시간이 많이 필요해 제외

1.3 AI와 가설을 다듬은 과정

1. 처음 질문은 “AI를 쓰면 코딩이 쉬워지는가?”처럼 너무 넓었다.
2. AI와 함께 완료 시간, 코드 이해도, 디버깅 횟수, 프롬프트 구체성처럼 숫자로 측정 가능한 후보로 나누었다.
3. “일주일 안에 데이터를 모을 수 있는가?”, “반대 결과도 의미가 있는가?”, “무엇으로 측정할 것인가?”를 기준으로 후보를 비교하였다.
4. 직접 실험보다 공개된 실험 데이터를 다시 계산하는 방식이 조건과 원자료를 남기기 쉬워 프롬프트 구체성 주제를 선택하였다.
5. 가설을 “자세히 물으면 더 좋다”에서 “문제 설명과 명시적 테스트가 포함된 고구체성 프롬프트에서 테스트 통과율이 높다”로 수정하였다.

1.4 최종 가설과 측정

최종 가설: 모호한 프롬프트보다 문제 설명과 명시적 테스트 예시가 포함된 구체적인 프롬프트에서 AI 생성 코드의 테스트 통과율이 높을 것이다.

영향을 주는 것: 프롬프트의 구체성(저구체성/고구체성).

영향을 받는 것: AI 생성 코드의 기능적 정확도. 기능적 정확도는 생성 코드가 원 연구의 테스트를 통과한 횟수를 전체 평가 횟수로 나눈 통과율(%)로 측정한다.

가설이 틀렸다면: 고구체성 조건의 통과율이 저구체성 조건과 같거나 더 낮고, 대부분의 모델에서 개선이 나타나지 않을 것이다.

데이터 범위: 공개 원 연구의 5개 모델 × 4개 프롬프트 유형 × 유형당 모델별 63회 집계 결과를 사용한다. 핵심 가설 비교는 저구체성 315회와 고구체성 315회, 총 630회 평가이다.

최종 선택 이유: IT 수업과 직접 연결되고, 공개 데이터로 재현할 수 있으며, 통과/실패라는 객관적 지표가 있고, 향후 웹사이트에서 표·그래프로 시각화하기 쉽기 때문이다.

2. 선행연구와 논거

문헌은 제목 검색으로 실재 여부를 확인하고, 저자·연도·출처와 초록의 핵심 결론을 원문 페이지에서 대조하였다. 숫자와 결론은 AI 요약만 사용하지 않고 원문 또는 공식 학술 페이지에서 다시 확인하였다.

문헌	출처	확인된 핵심 내용	본 가설과의 연결
Murr, Grainger, & Gao (2023)	arXiv 2311.07599	104개 코딩 문제와 네 가지 프롬프트 유형을 사용해 여러 LLM의 정확도·효율을 비교	본 과제의 재분석 데이터와 프롬프트 조건 정의를 제공
Shirafuji et al. (2022)	SAGE / DOI 10.3233/FAIA220264	문제 설명을 더 명시적으로 형식화했을 때 코드 생성 성능이 30.9%에서 39.9%로 증가	문제 설명 방식이 코드 생성 성능에 영향을 줄 수 있음을 뒷받침
Chen et al. (2021)	arXiv 2107.03374	HumanEval을 통해 실행 테스트로 생성 코드의 기능적 정확성을 평가	테스트 통과를 코드 정확도 지표로 사용하는 근거
Zi, Menon, & Guha (2025)	ACL Anthology / DOI 10.18653/v1/2025.ijcnlp-long.128	프롬프트를 최소 정보부터 상세 정보까지 단계화했으며 명시적 I/O와 엡지 케이스가 성능에 중요한 요소임을 보고	프롬프트 구체성이 항상 동일하게 작용하지 않으며 세부 정보의 종류가 중요함을 뒷받침

2.1 제외한 자료

개인 블로그, 프롬프트 예제 모음, 검증 절차가 없는 GitHub 튜토리얼은 실제 사용 팁에는 도움이 되지만 연구 설계와 데이터 검증이 불분명하여 핵심 선행연구 목록에서 제외하였다.

2.2 이미 알려진 사실과 이번 연구의 새 주장

이미 알려진 사실: LLM의 코드 생성 성능은 문제 설명과 프롬프트 구조에 민감할 수 있으며, 실행 테스트는 기능적 정확성을 측정하는 대표적인 방법이다(Chen et al., 2021; Shirafuji et al., 2022).

이번 연구의 새 주장: Murr et al.(2023)의 공개 집계 데이터를 동일한 계산 규칙으로 다시 합산했을 때, 저구체성 63.49%보다 고구체성 72.38%가 8.89%p 높지만 5 개 모델 중 1 개에서는 반대 방향이 나타났다. 따라서 “구체성이 대체로 도움이 되지만 모델에 따라 효과가 다르다”는 범위로 결론을 제한한다.

2.3 출처 확인 방법

6. 문헌 제목으로 검색해 실제 문헌이 존재하는지 확인했다.
7. 저자와 발표 연도를 공식 arXiv, SAGE, ACL Anthology 페이지와 대조했다.
8. 초록에서 연구 대상·방법·결론이 본문에 인용한 내용과 일치하는지 확인했다.
9. 원자료로 사용한 Murr et al.(2023)의 연구 방법과 프롬프트 유형 정의를 원문에서 확인했다.

3. 연구 방법: 실험 설계와 데이터

3.1 연구 설계

본 연구는 새로운 LLM 을 직접 실행하는 실험이 아니라 이미 공개된 코드 생성 실험의 집계 데이터를 재분석하는 2 차 분석이다. 공개 데이터를 사용해도 과제 조건을 충족하므로, 분석 전에 비교 조건, 표본 범위, 측정값과 계산식을 고정하였다.

구분	내용
바꾼 조건(독립변수)	프롬프트 유형. 핵심 비교는 저구체성 ‘함수명이 가려진 테스트만’ vs 고구체성 ‘문제 설명+명시적 테스트’
고정한 조건	동일한 5 개 모델 집합, 각 모델·프롬프트 유형의 전체 평가 수 63, pass/total 기반 계산식, 동일한 합산 규칙
표본/반복 범위	전체 데이터: 5 모델×4 유형×63=1,260 평가. 핵심 가설 비교: 5 모델×2 유형×63=630 평가
측정값	통과 수(pass), 전체 수(total), 통과율(%), 저구체성→고구체성 변화(%p)
원자료	source_data.csv
분석 절차	analysis.py 로 통과율·차이·모델별 변화를 계산하고 CSV·PNG 생성

3.2 공개 데이터 출처와 받은 날짜

출처: Murr, L., Grainger, M., & Gao, D. (2023). Testing LLMs on Code Generation with Varying Levels of Prompt Specificity. arXiv:2311.07599.

받은 날짜: 2026 년 9 월 15 일. 재현 패키지의 source_data.csv 에는 원 논문 집계표의 모델별 pass/total 값을 옮겼다.

3.3 실행 절차

10. 재현 ZIP 의 source_data.csv 를 확인한다.
11. Python 3 환경에서 pandas 와 matplotlib 을 설치한다.
12. 같은 폴더에서 python analysis.py 를 실행한다.
13. results_summary.csv, model_comparison.csv 와 두 그래프가 생성되는지 확인한다.
14. 출력값 저구체성 63.49%, 고구체성 72.38%, 차이 +8.89%p, 개선 모델 4/5 를 확인한다.

3.4 설계 변경 기록

주제 탐색 단계에서는 직접 AI를 여러 번 실행하는 실험도 검토했지만, 모델 버전·세션·온도 등의 조건을 동일하게 유지하기 어렵고 시간이 많이 필요해 데이터 수집을 시작하기 전에 공개 데이터 재분석 방식으로 확정하였다. 공개 데이터 분석을 시작한 뒤에는 결과에 맞추기 위해 비교 조건이나 계산식을 변경하지 않았다.

4. 결과

프롬프트 유형	통과	전체	통과율
모호한 테스트형	200	315	63.49%
문제 설명만	222	315	70.48%
테스트만	231	315	73.33%
문제 설명+테스트	228	315	72.38%

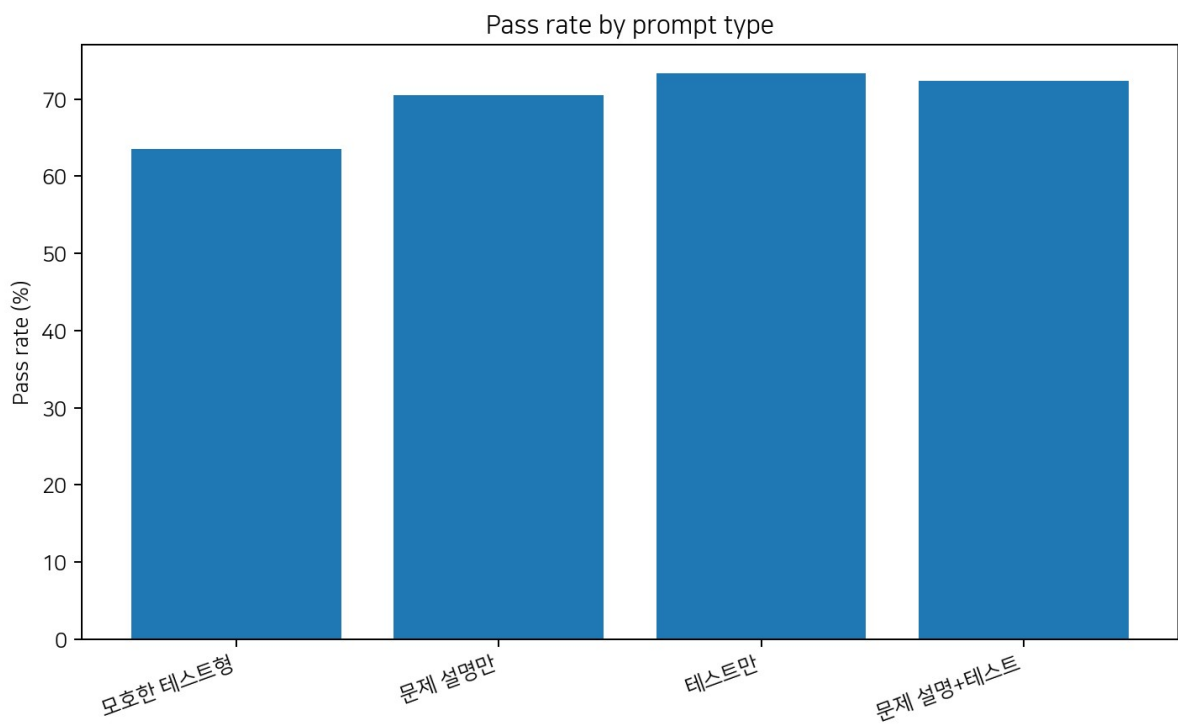


그림 1. 프롬프트 유형별 코드 통과율

가설의 핵심 비교에서 저구체성 조건은 $200/315=63.49\%$, 고구체성 조건은 $228/315=72.38\%$ 였다. 고구체성 조건이 8.89%p 높아 전체 집계에서는 가설과 같은 방향이 나타났다.

모델	저구체성	고구체성	변화
Bard	57.14%	73.02%	+15.87%p
Claude	69.84%	88.89%	+19.05%p
GPT-3.5	76.19%	87.30%	+11.11%p
GPT-4	80.95%	87.30%	+6.35%p
Text-davinci-002	33.33%	25.40%	-7.94%p

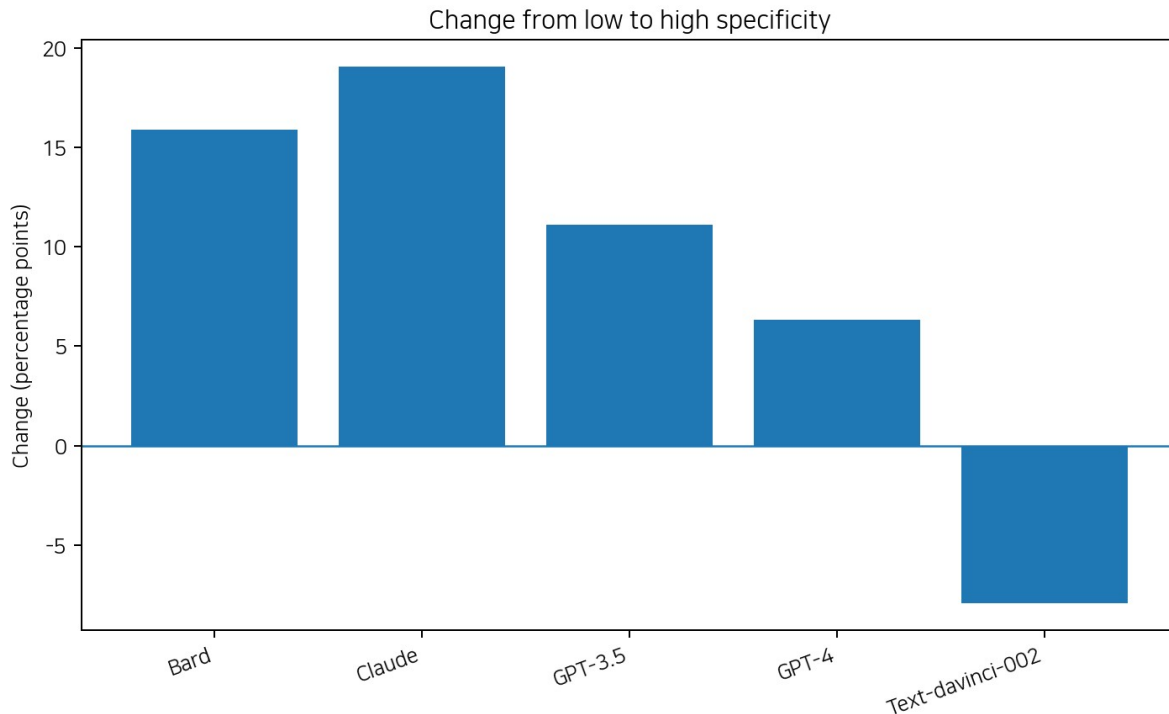


그림 2. 모델별 저구체성→고구체성 통과율 변화

4.1 가설대로 나온 부분

전체 통과율은 고구체성 조건이 8.89%p 높았고, Bard, Claude, GPT-3.5, GPT-4 네 모델에서 통과율이 증가했다. 이 값들은 source_data.csv 의 pass/total 합계와 analysis.py 계산 결과에서 다시 확인할 수 있다.

4.2 가설과 다르게 나온 부분

Text-davinci-002에서는 33.33%에서 25.40%로 7.94%p 감소했다. 또한 네 프롬프트 전체를 보면 ‘테스트만’ 조건의 전체 통과율 73.33%가 ‘문제 설명+테스트’ 72.38%보다 약간 높았다. 따라서 단순히 정보가 많을수록 항상 정확도가 높아진다고 말할 수 없다.

4.3 반대 결과의 가능한 이유

- 모델마다 문제 설명과 테스트 정보를 해석하는 능력이 달라 구체성의 효과가 다를 수 있다.
- 저구체성/고구체성 조건은 문장 길이뿐 아니라 함수명 공개 여부, 문제 설명, 테스트 정보가 함께 달라져 어떤 요소가 효과를 냈는지 분리하기 어렵다.
- 특정 모델은 예시 테스트에 과도하게 맞추거나 추가 정보에 의해 오히려 잘못된 추론을 할 수 있다.

4.4 가설 수정 기록

수정 전 생각: “프롬프트가 구체적일수록 모든 모델의 코드 정확도가 높아질 것이다.”

데이터 확인 후 최종 해석: “문제 설명과 테스트가 포함된 구체적인 프롬프트는 전체적으로 높은 통과율과 관련되지만, 모델과 프롬프트 구성에 따라 효과의 크기와 방향이 달라질 수 있다.”

5. 논의

본 분석은 프롬프트 구체성이 AI 코드 생성 결과에 영향을 줄 수 있다는 선행연구와 같은 방향을 보였다.

Shirafuji et al.(2022)은 명시적으로 형식화한 문제 설명에서 성능 향상을 보고했고, Zi et al.(2025)은 명시적 입출력과 엡지 케이스 같은 세부 정보가 프롬프트 구체성의 중요한 요소임을 보였다. 본 분석에서도 문제 설명과 테스트를 함께 제공한 조건이 가장 모호한 조건보다 전체적으로 높은 통과율을 보였다.

그러나 가장 상세해 보이는 조건이 네 조건 중 항상 최고는 아니었다. “테스트만” 조건이 전체 통과율에서 근소하게 높았고, Text-davinci-002 는 고구체성 조건에서 성능이 떨어졌다. 따라서 실무에서 중요한 것은 프롬프트를 무조건 길게 만드는 것이 아니라 요구사항, 입력과 출력, 오류 조건, 테스트 예시가 서로 일관되게 구성되도록 하는 것이다.

5.1 이 연구로 말할 수 없는 것

- 공개 집계 데이터만 사용했으므로 개별 문제에서 어떤 코드 오류가 발생했는지 설명할 수 없다.
- 사용된 모델 일부는 최신 모델이 아니므로 2026 년의 모든 AI 코딩 도구에 동일하게 일반화할 수 없다.
- 프롬프트 구체성과 코드 정확도의 관계를 보여줄 뿐, 구체성이 정확도를 직접 “원인”으로 만든다고 인과적으로 단정할 수 없다.
- 문제 설명, 함수명, 테스트 정보가 동시에 달라진 조건이 있으므로 각 요소의 독립 효과는 분리할 수 없다.

6. 결론

본 연구는 공개된 AI 코드 생성 실험 데이터를 이용하여 프롬프트 구체성과 생성 코드의 기능적 정확도 관계를 재분석하였다. 가장 모호한 테스트형 프롬프트의 통과율은 63.49%였고, 문제 설명과 명시적 테스트가 포함된 고구체성 프롬프트는 72.38%로 8.89%p 높았다. 5 개 모델 중 4 개에서 같은 방향의 개선이 나타나 최종 가설은 전체 수준에서 지지되었다. 그러나 Text-davinci-002 에서는 반대 결과가 나타났고, 네 조건 전체에서는 ‘테스트만’ 조건이 근소하게 가장 높았다. 따라서 결론은 “구체적인 프롬프트가 대체로 도움이 되지만 항상 그렇지 않다”로 제한해야 한다. AI 를 활용한 코딩에서는 사용할 언어, 기능, 입력·출력, 예외 조건과 테스트를 명확히 제공하고, 생성된 코드를 실제 테스트로 확인하는 것이 가장 실용적인 사용 원칙이다.

7. 재현 방법

제출 ZIP 안의 repro 폴더에 원자료(source_data.csv), 실행 스크립트(analysis.py), 결과 CSV, 그래프, README 가 함께 들어 있다.

15. ZIP 을 새 폴더에 푼다.

16. repro 폴더에서 pandas 와 matplotlib 을 설치한 뒤 python analysis.py 를 실행한다.

17. 터미널에 63.49%, 72.38%, +8.89%p, 4/5 가 출력되는지 확인한다.

18. 결과 CSV 2 개와 그래프 PNG 2 개가 생성되는지 확인한다.

8. AI 와 나의 판단 3 줄

1. AI 에게 맡긴 일: 후보 주제를 비교하고 가설을 측정 가능한 문장으로 좁히며, 공개 선행연구를 찾고 공개 집계 데이터의 통과율·표·그래프를 계산하게 했다.
2. 내가 직접 판단한 일: IT 수업과 향후 웹 프로젝트로 확장하기 쉬운 ‘프롬프트 구체성과 코드 정확도’를 최종 주제로 선택하고, 결과가 실제 원자료와 맞는지 확인했다.
3. AI 제안을 따르지 않은 일: 의료 데이터 주제와 ‘프롬프트가 자세할수록 항상 좋아진다’는 단정은 제외하고, 5 개 모델 중 1 개에서 반대 결과가 나온 사실도 그대로 결론에 반영했다.

9. 참고문헌

1. Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. <https://arxiv.org/abs/2107.03374>
2. Murr, L., Grainger, M., & Gao, D. (2023). Testing LLMs on Code Generation with Varying Levels of Prompt Specificity. arXiv:2311.07599. <https://arxiv.org/abs/2311.07599>
3. Shirafuji, A., Ito, T., Morishita, M., Nakamura, Y., Oda, Y., Suzuki, J., & Watanobe, Y. (2022). Prompt Sensitivity of Language Model for Solving Programming Problems. New Trends in Intelligent Software Methodologies, Tools and Techniques. <https://doi.org/10.3233/FAIA220264>
4. Zi, Y., Menon, H., & Guha, A. (2025). More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation. Proceedings of IJCNLP-AACL 2025, 2380-2402. <https://doi.org/10.18653/v1/2025.ijcnlp-long.128>

10. 과제 완주 체크리스트 T10-C01~C51

기준	확인 내용	상태
T10-C01	관심 분야 명시	완료
T10-C02	후보 주제 3 개 이상	완료
T10-C03	후보별 제외 이유	완료
T10-C04	최종 가설 한 문장	완료
T10-C05	무엇이 무엇에 영향인지 명시	완료
T10-C06	가설 표현의 측정 방법	완료
T10-C07	가설이 틀릴 때 결과	완료
T10-C08	데이터 범위 명시	완료
T10-C09	AI 와 가설을 다듬은 과정	완료
T10-C10	최종 가설 선택 이유	완료
T10-C11	참고문헌 목록	완료
T10-C12	저자와 연도	완료
T10-C13	학술지·기관·URL 출처	완료
T10-C14	문헌별 가설 연결 설명	완료
T10-C15	제외 문헌과 이유	완료
T10-C16	실험/데이터 검증 연구 포함	완료
T10-C17	알려진 사실과 새 주장 구분	완료
T10-C18	본문 인용과 참고문헌 일치	완료
T10-C19	참고문헌 형식 통일	완료
T10-C20	출처 확인 방법	완료
T10-C21	실험/분석 설계 표	완료

T10-C22	바꾼 조건·고정 조건 구분	완료
T10-C23	반복 횟수/표본 수	완료
T10-C24	측정값 명시	완료
T10-C25	원자료 파일 포함	완료
T10-C26	원자료 건수와 설계 범위 일치	완료
T10-C27	공개 데이터 출처와 받은 날짜	완료
T10-C28	실행 절차/스크립트 포함	완료
T10-C29	짧은 재현 확인 방법	완료
T10-C30	설계 변경 여부와 이유 기록	완료
T10-C31	결과 표/그래프	완료
T10-C32	가설대로/다르게 나온 부분 구분	완료
T10-C33	가설 지지 판단	완료
T10-C34	판단 근거 원자료 값 제시	완료
T10-C35	가설과 다른 결과도 포함	완료
T10-C36	반대 결과 가능성 검토	완료
T10-C37	가설 수정 전·후 기록	완료
T10-C38	최종 결론 한 문단 이상	완료
T10-C39	연구 한계	완료
T10-C40	원자료에 없는 주장 배제	완료
T10-C41	표지 제목·작성자·작성일	완료
T10-C42	초록	완료
T10-C43	초록에 문제·방법·결과 포함	완료
T10-C44	서론·방법·결과·논의 순서	완료
T10-C45	참고문헌 본문 뒤 배치	완료
T10-C46	재현 방법	완료
T10-C47	원자료와 실행 절차 함께 포함	완료
T10-C48	최종 제출용 ZIP 구성	완료
T10-C49	AI 말긴 일과 직접 판단 구분	완료
T10-C50	미완성 표시/자리표시자 없음	완료
T10-C51	제출 URL 없음(로그인 문제 없음)	완료

최종 확인: 본 문서와 재현 폴더에는 [직접 입력 필요], TODO, 자리표시자 같은 미완성 표시가 없으며, 제출에 필수적인 외부 로그인 URL 은 사용하지 않는다.